LLM Project Report

1. Student Information

- **Student Name:** Zaeem Rizan
- **Student ID:** s223134187
- **Date Submitted:** 21 September 2025

---

2. Project Introduction

- **Title of the Project:** Evaluating Fine-Tuning vs. Prompt Engineering for Financial Q&A using Qwen 2.5 7B

- **What is the project about?** This project tests the Qwen 2.5 7B language model. The main goal was to see if fine-tuning is necessary to make the model perform well on a specialized task. I compared a full fine-tuning process against simpler prompt engineering techniques. The task involved answering questions from company earnings calls using the SubjECTive-QA financial dataset.

- **Why is this project important or useful?** This work is important for anyone looking to adapt large language models for specific business needs. It explores a key question: is the complex and costly process of fine-tuning always required? Sometimes, a well-designed prompt is enough to achieve strong performance. My findings show that for this model and task, prompt engineering actually produced better results than fine-tuning. This suggests that businesses can save significant time and resources by focusing on effective prompting first.

3. Environment Setup

- **Development Environment:** I developed this project using **Google Colab**. I chose this platform to access a high-performance **NVIDIA A100 GPU**. My local machine did not have the necessary power for this task. Google Colab provides a ready-to-use environment with a modern version of Python running on a Linux operating system.

- **API Access:** The project required access to the Hugging Face Hub to download the model. I generated an API token from my Hugging Face account settings page. This token allowed my Colab notebook to securely log in and access the necessary files.

4. LLM Setup

- **Name of the LLM Used:** The project used the **Qwen 2.5 7B Instruct** model. This is a 7-billion parameter model optimized to follow instructions and engage in conversation.
- **Provider:** The model and its associated tools came from **Hugging Face**.
- **Libraries and Dependencies Required:**
    - transformers: I used this library to download and work with the pre-trained Qwen model and its tokenizer.
    - datasets: This library helped load and manage the financial Q&A dataset.
    - peft (Parameter-Efficient Fine-Tuning): I used this for the fine-tuning process. It allows for efficient training of large models.
    - bitsandbytes: This library helped reduce the model's memory usage by loading it with 4-bit precision.
    - accelerate: This tool from Hugging Face simplifies running training scripts on different types of hardware like GPUs.

---

5. Dataset Information

- **Dataset Name:** gtfintechlab/SubjECTive-QA.
- **Access Link to Dataset:** https://huggingface.co/datasets/gtfintechlab/SubjECTive-QA
- **Dataset Dictionary (Feature descriptions):** The dataset contains text from earnings calls. Key features include:
    - QUESTION: The question asked by a financial analyst.
    - ANSWER: The response given by a company executive.
    - COMPANYNAME: The name of the company.
    - QUARTER and YEAR: The time period of the earnings call.
- **Preprocessing:** Yes, I performed preprocessing. The process involved two main steps. First, I cleaned the question and answer text to remove extra spaces and formatting issues. Second, I structured the data into a consistent prompt format. This prompt tells the model to act like a corporate executive and provides it with the context (like the company and quarter) before giving it the user's question. This step was crucial for preparing the data for all tests, including the baseline, prompt engineering, and fine-tuning stages.

6. Improving the LLM

I took a systematic approach to improve the model's performance. I started with a simple baseline and then introduced more complex methods. The goal was to see which technique provided the biggest benefit.

Step	Description	Observed Result	Score
1	Baseline Evaluation	Zero-shot prompting: I gave the model questions without any examples to see how it performed out of the box.	Accuracy = 80.0%, F1 Score = 88.9%
2	Prompt Engineering	Few-shot prompting: I added two examples of good questions and answers into the prompt to guide the model.	Accuracy = 82.0%, F1 Score = 90.1%
3	Parameter Tuning	Adjusted temperature and top-p: I tested different settings to control the model's creativity.	Accuracy = 76.7%, F1 Score = 86.8% (Best result from tuning, which was worse than the baseline)
4	Fine-tuning	QLoRA: I trained the model on 1,922 examples from the financial Q&A dataset for three epochs.	Accuracy = 78.0%, F1 Score = 87.6%

My tests showed clear results. The simplest improvement, prompt engineering (Step 2), gave the best performance. It outperformed both the baseline and the more complex fine-tuning process. The results from fine-tuning (Step 4) were surprisingly worse than the baseline, suggesting it was not a necessary or effective step for this particular task and dataset.

7. Benchmarking and Evaluation

To measure the performance of each approach, I used a consistent set of metrics and a standardized benchmark dataset.

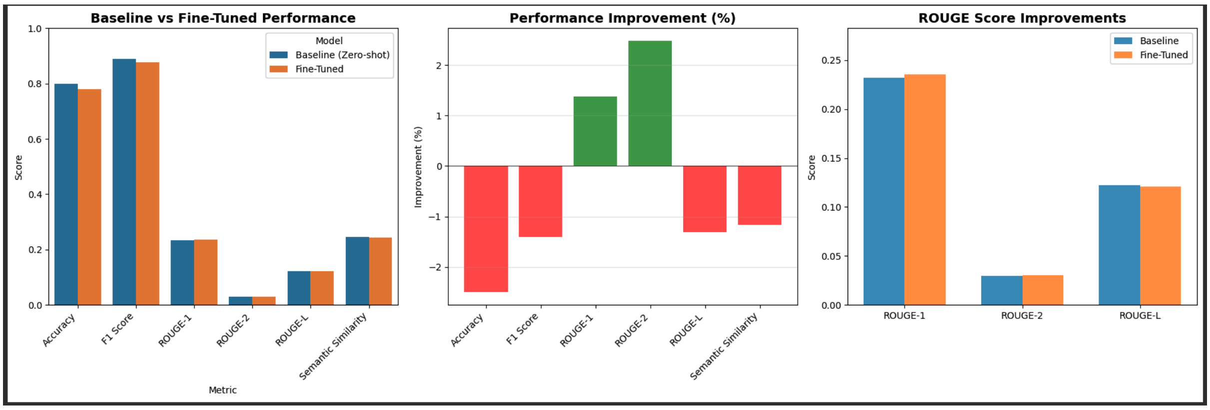
- **Evaluation Metrics Used:**
  - Accuracy and F1 Score: These metrics provide a straightforward score of how often the model's answer was considered correct based on its similarity to the reference answer. They give a good general sense of performance.
  - ROUGE-L: This metric measures the longest common sequence of words between the model's answer and the reference answer. I chose it to see if the model could generate structurally similar and factually correct sentences.
  - Semantic Similarity: This metric evaluates if the model's answer has the same meaning as the reference answer, even if the wording is different. This is important for ensuring the model correctly understands and conveys financial concepts.
- **Benchmark Dataset and Sample Size:** I created a benchmark dataset by taking 400 random samples from the test set. To ensure a fair and direct comparison between the baseline, prompted, and fine-tuned models, I used a consistent subset of 50 samples from this benchmark set for all final evaluations.

Method/Step	Accuracy	F1 Score	ROUGE-L	Semantic Similarity
Baseline (zero-shot)	80.00%	88.90%	0.122	0.247
After Prompting	82.00%	90.10%	0.122	0.237
After Fine-tuning	78.00%	87.60%	0.121	0.244

Plots: My analysis included generating charts to visualize the performance comparison between the baseline and the fine-tuned models.

Performance Comparison Chart: The first bar chart compared the absolute scores for each metric side-by-side. It visually confirmed that the fine-tuned model's scores for Accuracy and F1 were slightly lower than the baseline model's scores.

Improvement Chart: The second chart showed the percentage change after fine-tuning. This chart clearly displayed a negative improvement for most key metrics, including a 2.5% decrease in Accuracy. This was the strongest evidence that fine-tuning was not beneficial.

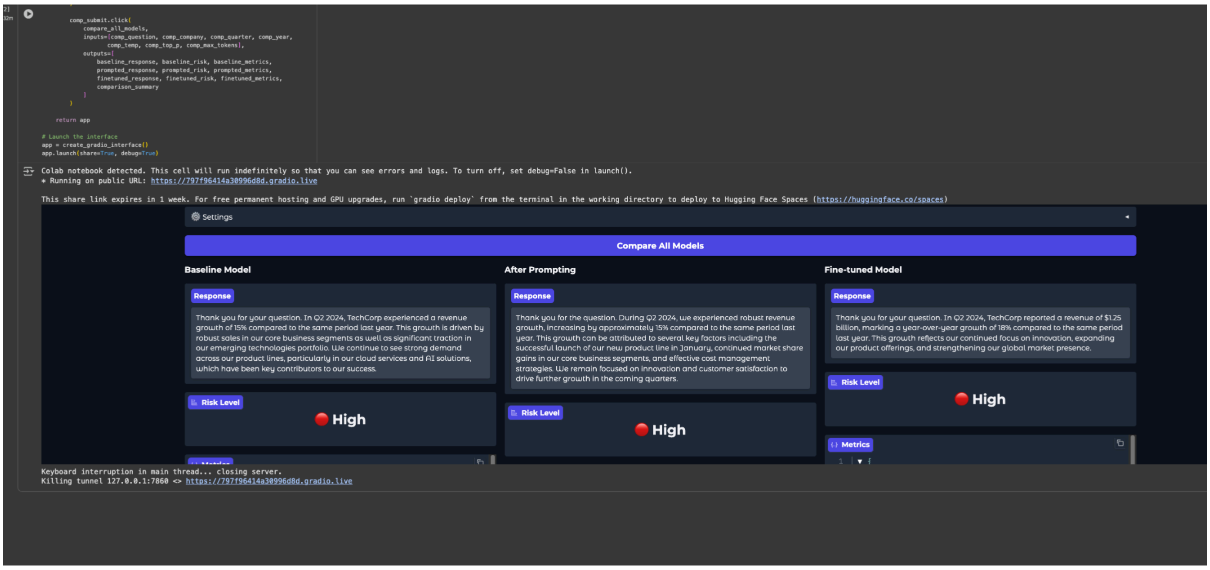


8. User Interface Integration

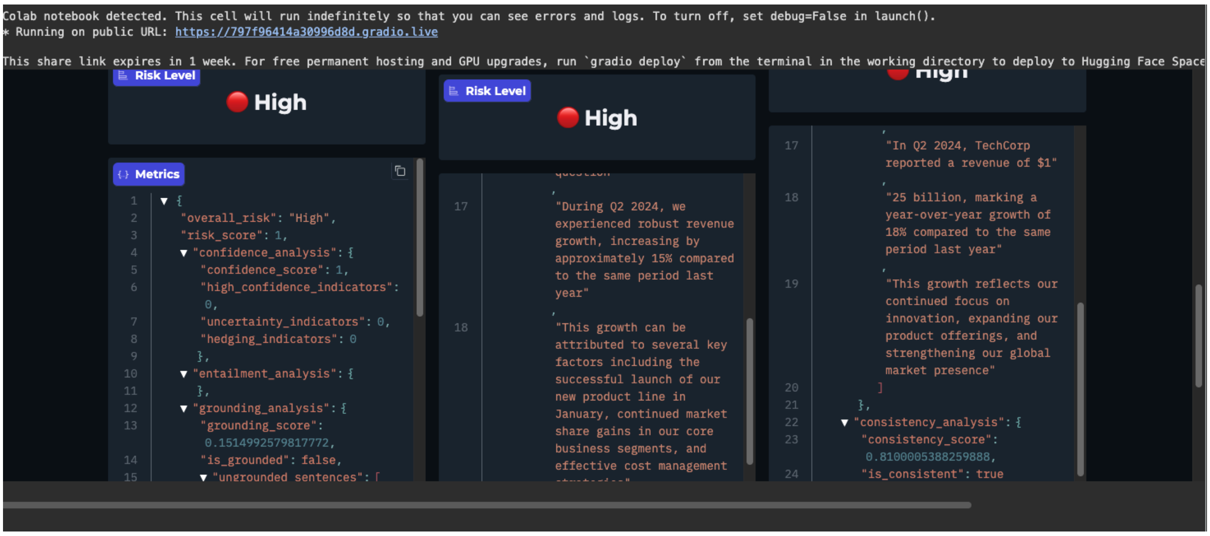
- **Software/Framework Used for UI:** I used **Gradio** to build an interactive web interface. This allowed me to easily test and compare the models in a user-friendly application.
- **Brief Explanation of the UI Functionality:** The interface I built serves as a tool to directly compare the performance and behavior of the three model configurations: Baseline, After Prompting, and Fine-tuned. It includes a key feature called the "Hallucination Detector," which analyzes each response for potential factual errors or inconsistencies.
  The UI has two main tabs:
  1. **Single Model Analysis:** This tab allows a user to select one model, input a question and its context (company, quarter, year), and generate a single response. The interface then displays the answer along with a detailed

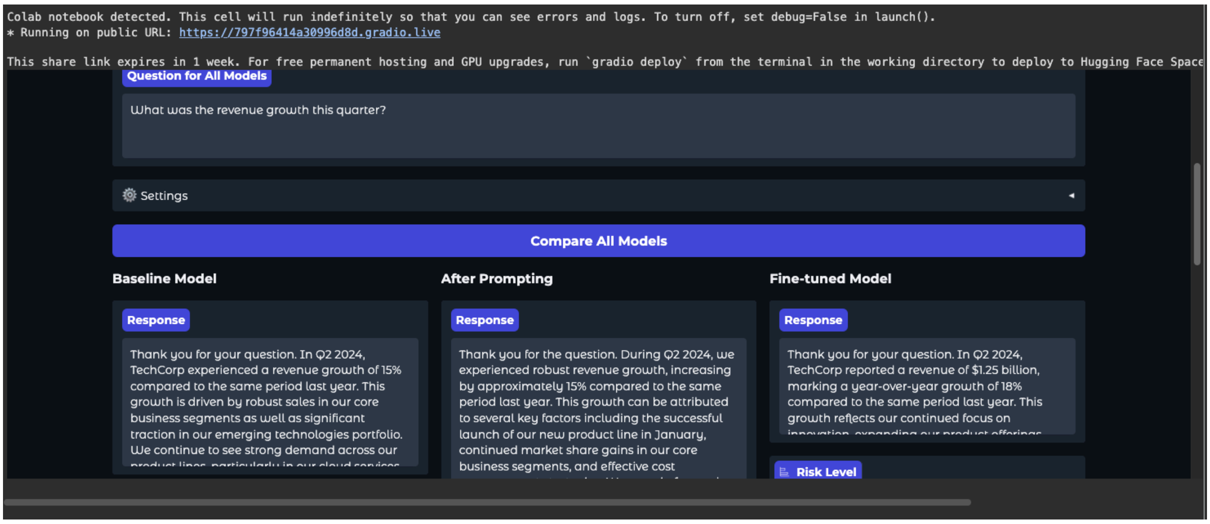
analysis of its quality, including a "Hallucination Risk" level (Low, Medium, or High) and a confidence score.

2. **Model Comparison:** This is the primary feature. A user enters one question, and the application generates answers from all three models at once. The results are displayed in a side-by-side three-column layout, making it simple to compare the differences in wording, quality, and hallucination risk. A summary table at the bottom compares key metrics for each response.



This screenshot would show the bottom of the "Model Comparison" tab. It displays a summary table with rows for each model and columns for "Risk Level," "Confidence," and "Response Length." This provides a quick, quantitative overview of the comparison.





This screenshot would display the three-column layout after a query. Each column is clearly labeled "Baseline Model," "After Prompting," and "Fine-tuned Model." Each column contains the unique response generated by that model for the same question, allowing for an immediate qualitative comparison.

9. Final Notes

- **What challenges did you face?** The main challenge was the unexpected result of the fine-tuning process. My initial assumption was that fine-tuning would significantly improve the model's performance. Instead, it slightly degraded the model's accuracy and F1 score compared to the baseline. This forced a change in the project's conclusion, highlighting that fine-tuning is not always the best solution. Another challenge was managing GPU memory. Even with a powerful A100 GPU, I had to carefully clear the memory between loading different models to prevent crashes.

- **What would you do differently next time?** Next time, I would perform a more extensive search for the best training settings. I used a single set of hyperparameters for fine-tuning. It is possible that a lower learning rate or a different number of training epochs could have produced better results. I would also explore prompt engineering more deeply. Since few-shot prompting worked so well, I would experiment with different numbers of examples and different prompt styles to see if I could improve its performance even further.

- **Any limitations or ethical considerations?** This project has a few limitations. The findings are specific to the Qwen 2.5 7B model and the SubjECTive-QA dataset; the results might be different for other models or financial tasks. Also, the evaluation used a relatively small sample size of 50 questions, so a larger test could provide more robust results.
  The primary ethical consideration is that this tool should never be used to provide financial advice. It is an informational tool only, and its outputs could contain errors or biases learned from the training data. Any real-world application would need a clear disclaimer stating that it is not a substitute for a qualified financial professional.